

Análisis de Algoritmos I

Algoritmos de ordenamiento

María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Algoritmos de ordenamiento

- **Objetivo:** Poner elementos de una lista en una secuencia dada por una **relación de orden**, es decir, la salida ha de ser una permutación (o reordenamiento) de la entrada que satisfaga la relación de orden dada.
- Las relaciones más comunes son: el orden **numérico** y el orden **lexográfico**, ya sea ascendente o descendente.
- Ordenar de forma eficiente es importante para optimizar la eficiencia de otros algoritmos que requieren de listas ordenadas (como los de búsqueda y fusión).

Algoritmos de ordenamiento

- **Antes de las computadoras:** ordenar a mano

- Ordenar cartas
- Archivos en papel
- Listas telefónicas
- Diccionarios
- ...

- De ahí surgen ideas naturales como:

- comparar vecinos
- insertar en el lugar correcto

- **Años 50:** con las primeras computadoras nacen los primeros algoritmos “formales”

- ¿Cómo ordenamos datos que sea fácil de entender y fácil de programar?



Insertion sort



Bubble sort

Algoritmos de ordenamiento

- **Años 60:** la eficiencia importa
 - Las computadoras empiezan a manejar más datos y ordenar mal es costoso.
 - Uno de los primeros algoritmos eficientes, diseñado por **John von Neumann**, pensando en las cintas magnéticas y memoria limitada.
 - Divide el problema en partes iguales.
- **Años 60 – 70:** rapidez práctica
 - Algoritmo simple pero muy rápido en la práctica
 - Desarrollado por **Tony Hoare**, estudiante de universidad



Merge sort



Quick sort

Algoritmos de ordenamiento

- **Años 70 – 80:** se establece un límite teórico
 - No existe un algoritmo de ordenamiento por comparación mejor que $O(n \log n)$
 - El objetivo ya no es ser más rápidos, sino:
 - usar menos memoria
 - ser estable
 - aprovechar el orden previo
 - adaptarse al hardware
- **Años 90 – Presente:** algoritmos híbridos
 - Los lenguajes actuales no usan un solo algoritmo.
 - Python: Timsort → mezcla insertion + merge
 - Java / C++: quick sort + heap sort



Heap sort
Intro sort



Tim sort
Radix sort
Count sort

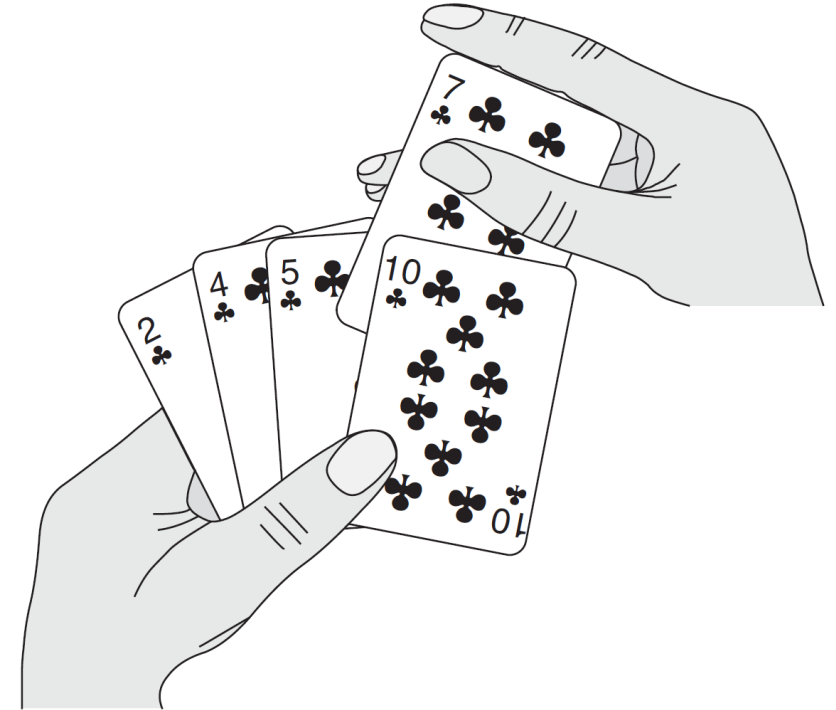
Algoritmos de ordenamiento

- Los números que queremos ordenar se les llama **llaves** (**keys o sort keys**).
- Usualmente vienen acompañados de otros datos, información asociada a las llaves, a la cual se le llama **datos satélite** (**satellite data**) que deben moverse junto con la llave.
 - Ejemplo: cuando ordenamos una columna de una tabla en Excel y te pregunta si se va extender el ordenamiento al resto de las columnas.
- Visualizaciones:
 - <https://visualgo.net/en/sorting>
 - <https://es.khanacademy.org/computing/computer-science/algorithms/insertion-sort/a/insertion-sort-pseudocode>

Insertion sort

(Ordenamiento por inserción)

- Analiza de izquierda a derecha, uno por uno, los datos de una sucesión.
- Una vez que se ha analizado cada dato, se inserta en un sitio idóneo en una sucesión ya ordenada.
- Es eficiente para listas pequeñas o cuando la lista está casi ordenada.



Insertion sort: Algoritmo

11, 7, 14, 1, 5, 9, 10.



Sucesión ordenada

11

7, 11

7, 11, 14

1, 7, 11, 14

1, 5, 7, 11, 14

1, 5, 7, 9, 11, 14

1, 5, 7, 9, 10, 11, 14

Sucesión no ordenada

7, 14, 1, 5, 9, 10

14, 1, 5, 9, 10

1, 5, 9, 10

5, 9, 10

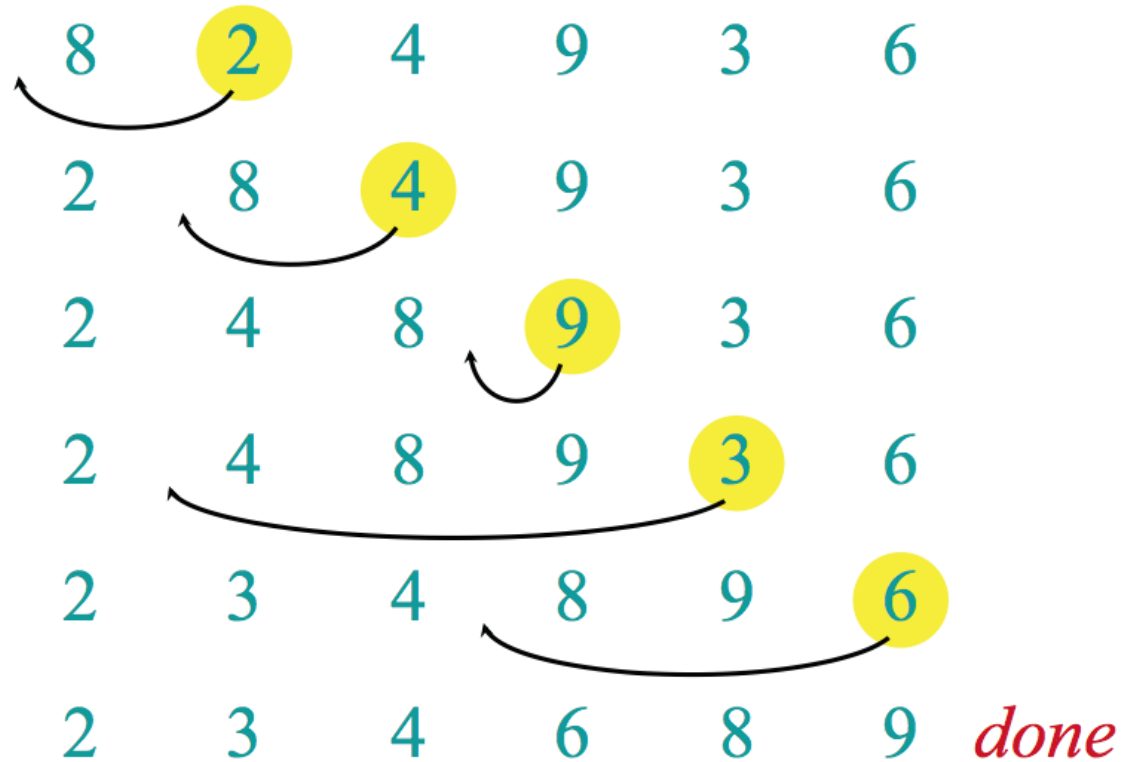
9, 10

10

Ejemplo
Insertion
Sort

Insertion sort: Ejemplo

$A=[8,2,4,9,3,6]$



Ejemplo
Insertion
Sort

Insertion sort: pseudocódigo

Algoritmo #3

Input: $x_1, x_2, \dots, x_n$

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

for $j = 1$ to n **do**

$i = j - 1$ #izq

$x = x_j$ #der #n-1 veces

while $x < x_i$ and $i \geq 0$ **do**

$x_{i+1} = x_i$ #despl. a la der p/ abrir espacio #por c/iter j: d_i

$i = i - 1$

$x_{i+1} = x$ #insertar a la izq #n-1 veces

Insertion sort: Análisis

En este análisis, tomaremos el **número de intercambios de datos** como lo que va a medir la complejidad temporal del algoritmo:

$$\textbf{(a)} \ x = x_j \quad \textbf{(b)} \ x_{i+1} = x_i \quad \textbf{(c)} \ x_{i+1} = x$$

Tenemos dos ciclos: uno exterior (**for**) y otro interior (**while**). En el ciclo exterior siempre se ejecutan dos operaciones de intercambio de datos: (a) y (c). Debido a que el ciclo interior puede o no ser ejecutado, el número de intercambios de datos realizados para x_j en el ciclo interior se denotará por d_j .

El número total de movimientos de datos para el ordenamiento por inserción directa es:

$$\begin{aligned} X &= \sum_{i=2}^n (2 + d_i) \\ &= 2(n-1) + \sum_{i=2}^n d_i \end{aligned}$$

Insertion sort: Mejor caso

- Cuando los datos de entrada **ya están ordenados**.
- El algoritmo nunca ejecuta el ciclo interior *while*.
- Únicamente se recorren las n posiciones del arreglo para confirmar que la información de cada casilla del arreglo está en la posición donde tiene que quedar.
- Número de intercambios:

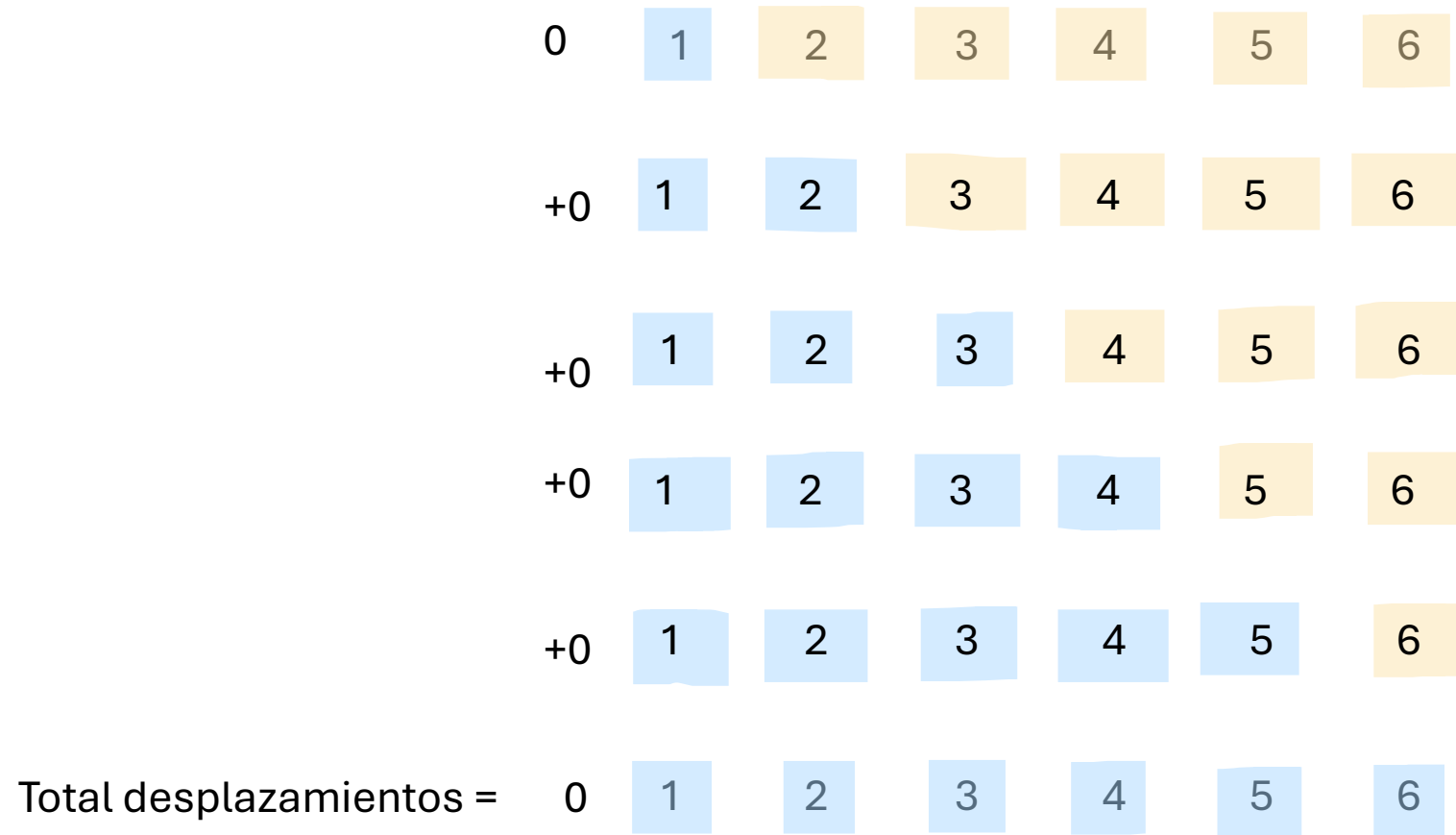
$$\sum_{i=2}^n d_i = 0,$$

$$A = [1, 2, 3, 4, 5, 6]$$

$$X = 2(n - 1)$$

- **Complejidad:** $O(n)$

Insertion sort: Mejor caso



En el mejor caso para $n=6$, el total sería 0 desplazamientos; es decir, se realizan $n-1$ comparaciones y ningún desplazamiento.

Insertion sort: Peor caso

- Cuando los datos de entrada están **ordenados a la inversa**.
- El ciclo *while* se ejecuta el **número máximo posible de iteraciones**: una vez para el elemento de la posición dos, se ejecuta dos veces para el elemento de la posición tres, se ejecuta tres veces para el elemento de la posición cuatro, y así sucesivamente hasta llegar a la posición n donde el ciclo se tiene que ejecutar $n-1$ veces.

- Número de intercambios:

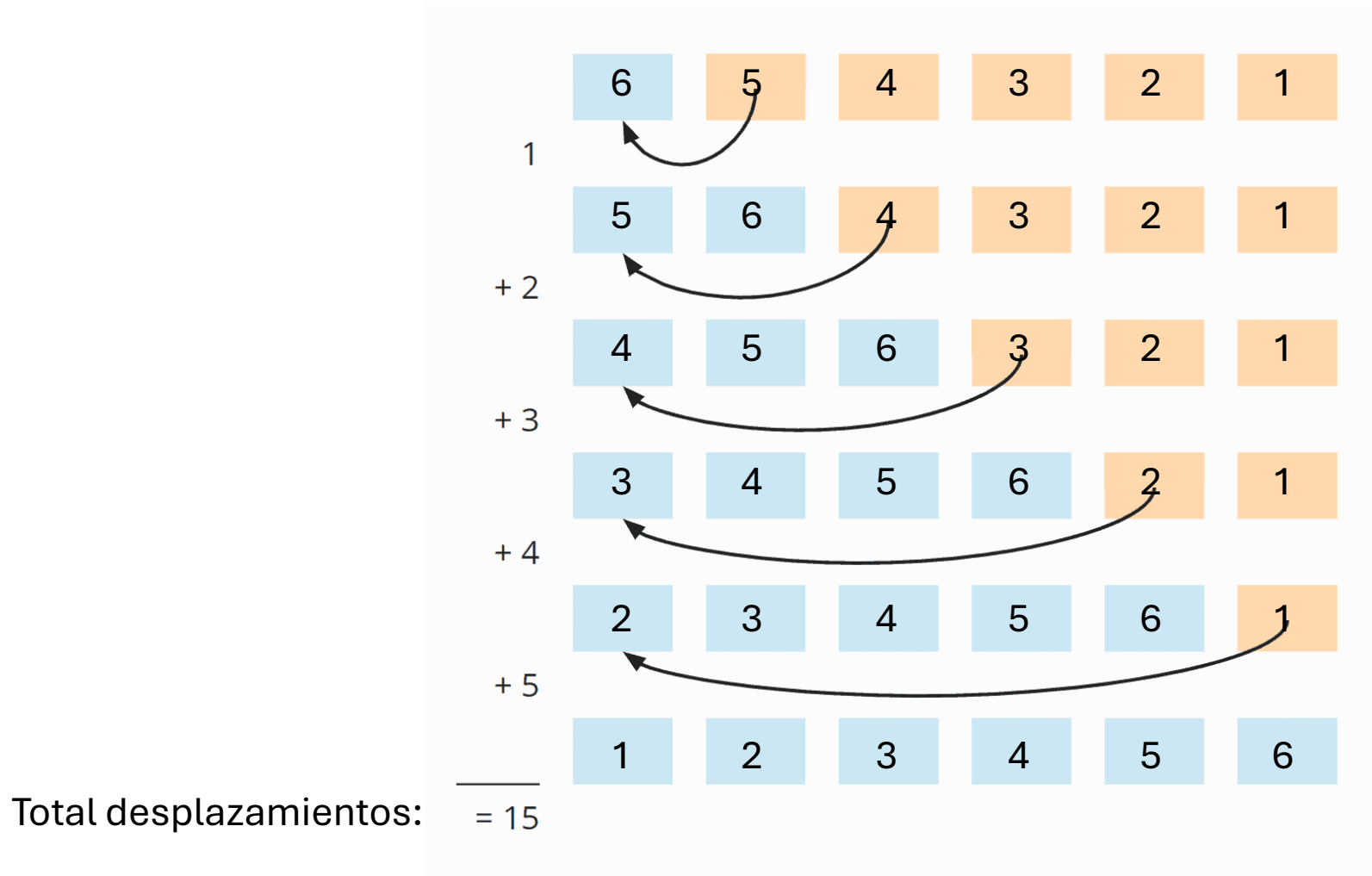
$A = [6, 5, 4, 3, 2, 1]$

Iteración (j)	Desplazamientos (d_j)	
2	1	$d_2 = 1$
3	2	$d_3 = 2$
4	3	$\vdots$
...	...	$\vdots$
n	n-1	$d_n = n - 1.$

$$= \frac{n(n-1)}{2} \approx \frac{n^2}{2}$$

- Si tenemos n elementos y cada uno se compara con una cantidad de elementos que depende de n , terminas haciendo aproximadamente **$n \times n$ operaciones**.
- **Complejidad:** $O(n^2)$

Insertion sort: Peor caso



Cada elemento nuevo tiene que compararse con prácticamente todos los que ya estaban ordenados.

Coincide con la lógica de que el número de desplazamientos $= \frac{n(n-1)}{2} = \frac{6(6-1)}{2} = \frac{6 \cdot 5}{2} = \frac{30}{2} = 15$.

Insertion sort: Caso promedio

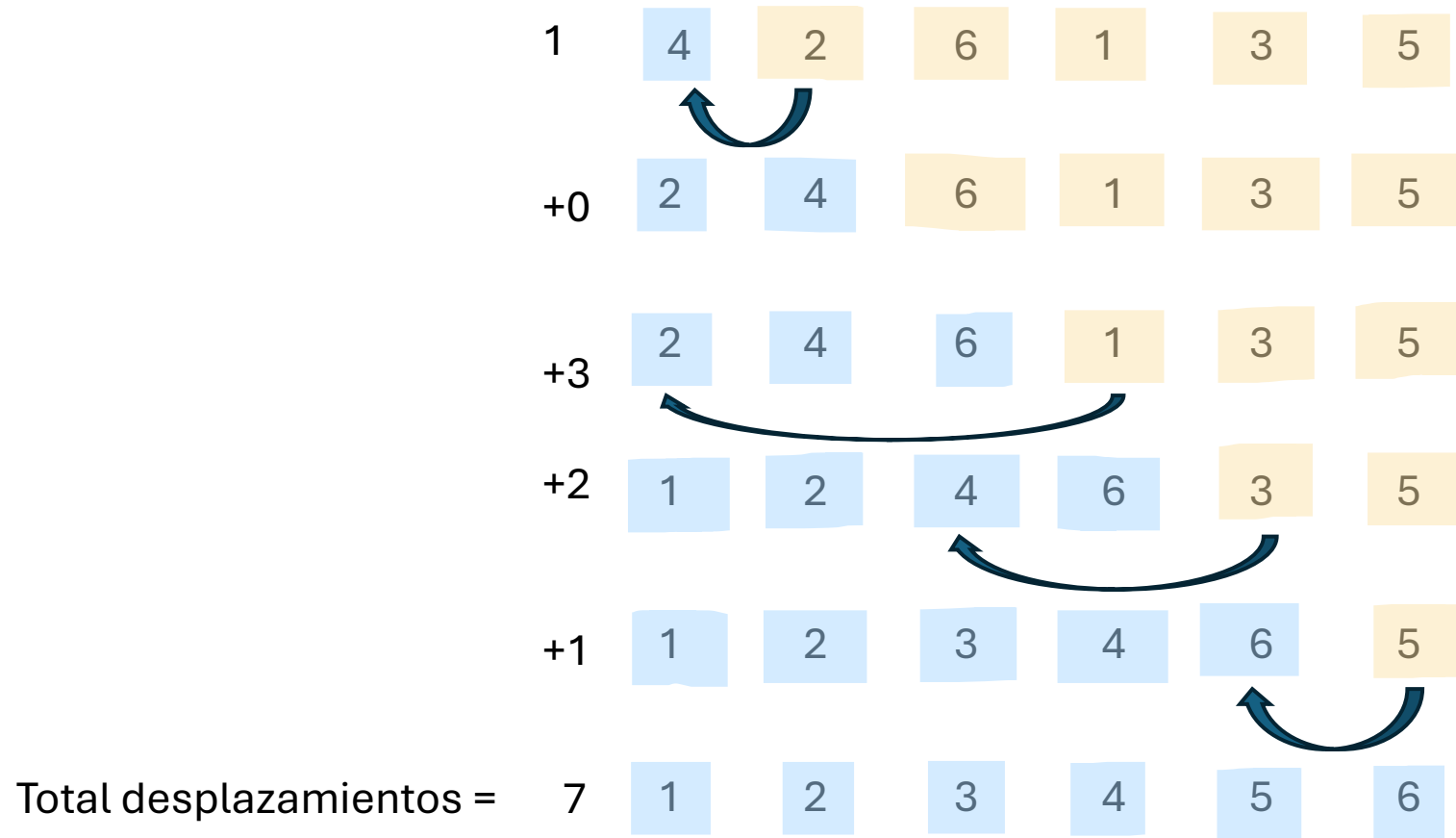
- Cuando los datos de entrada están en **orden aleatorio**, y en promedio, cada elemento termina moviéndose **la mitad de lo que podría moverse**.
- El ciclo interior se ejecutará aproximadamente **la mitad** de comparaciones.

- Número de comparaciones: $\approx \frac{n^2}{4}$

A = [4, 2, 6, 1, 3, 5]

- En promedio, haces **la mitad del trabajo que en el peor caso**, pero como ese trabajo sigue dependiendo de cuántos elementos ya ordenaste, el crecimiento sigue siendo cuadrático.
- **Complejidad:** $O(n^2)$

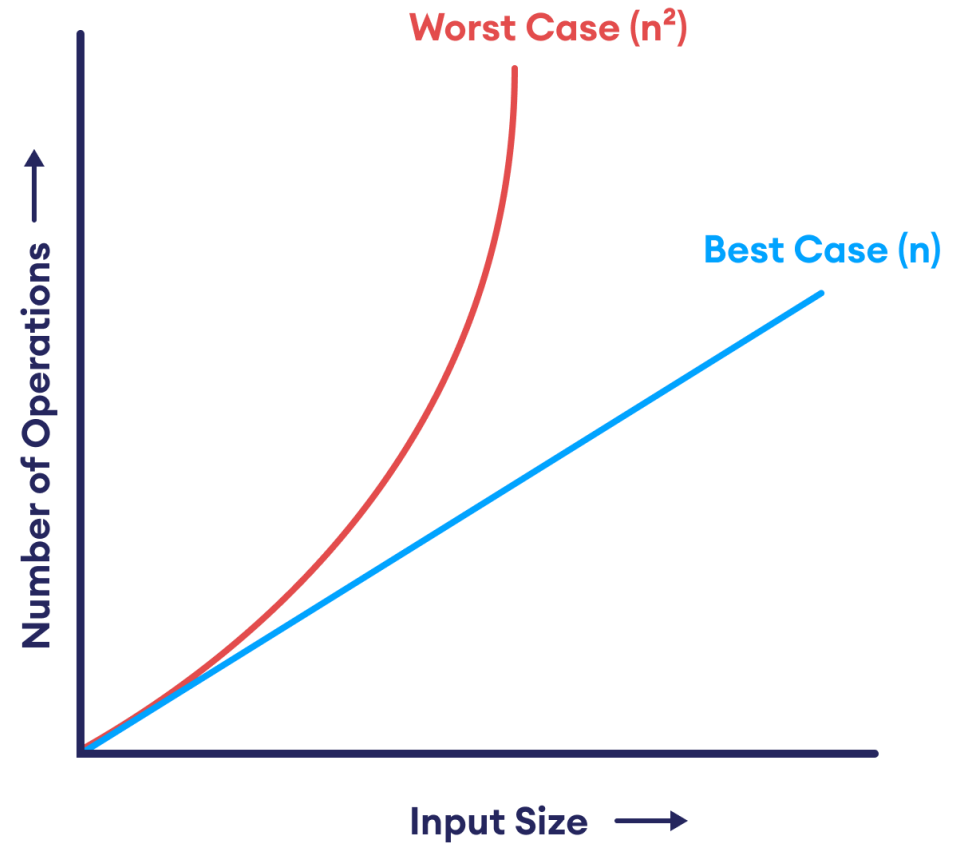
Insertion sort: Caso promedio



En el peor caso para $n=6$, el total sería 15 desplazamientos; aquí estamos cerca de la mitad, cumpliendo la lógica del promedio.

Insertion sort: Resumen

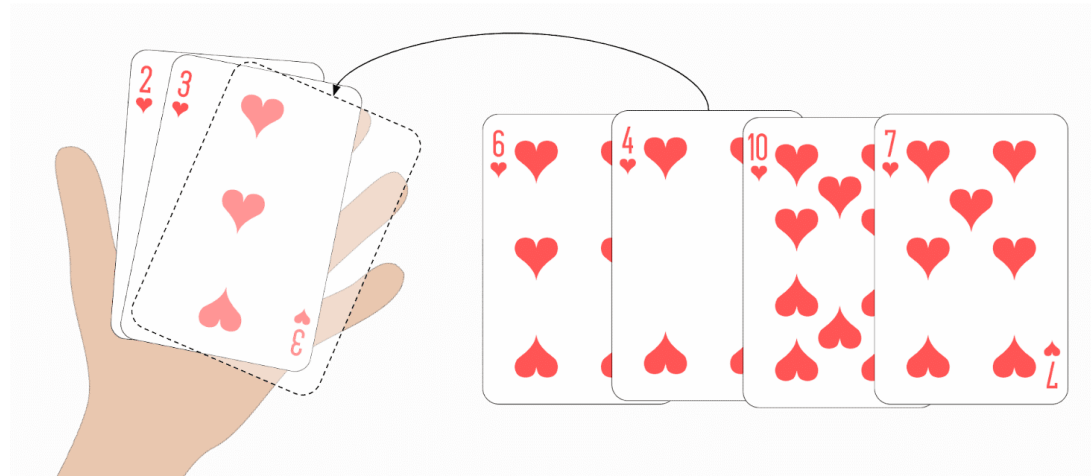
- **Mejor caso:** $O(n)$
- **Peor caso:** $O(n^2)$
- **Caso promedio** $O(n^2)$



Selection sort

(Ordenamiento por selección directa)

- Se encuentra el número más pequeño de un arreglo A , y si es necesario, se intercambia de tal manera que ocupe la posición a_1 del arreglo.
- Se repite el paso anterior, en busca del segundo número más pequeño y se coloca en la posición a_2 .
- El proceso continúa hasta que se encuentra el número más grande.
- Puede ser útil en listas pequeñas o cuando hay un entorno de memoria limitado. Pero es muy ineficiente con grandes cantidades de datos.



Selection sort: pseudocódigo

Algoritmo #4

Input: $x_1, x_2, \dots, x_n$

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

```
for  $i = 1$  to  $n-1$  do                                #n-1 veces
    pos_min = i                                         #pivote para indicar el mínimo

    for  $j = i+1$  to  $n$  do                                #n-i veces en cada iteración
        if  $x[j] < x[pos\_min]$  then
            pos_min = j                                #se actualiza la posición min
    temp = x[j]                                         #se intercambia
    x[j] = x[pos_min]
    x[pos_min] = temp
```

Selection sort: Análisis

Tenemos dos operaciones importantes: (a) la comparación de dos elementos y (b) el cambio del pivote:

$$\textbf{(a)} \ x[j] < x[\text{pos_min}] \qquad \textbf{(b)} \ \text{pos_min} = j$$

Ambas operaciones tendrán un tiempo de ejecución constante $O(1)$. Por lo tanto, la clave para analizar este algoritmo está en contar el **número de iteraciones que realiza el bucle interno**:

- Cuando $i = 1$, el bucle interno j va desde 2 hasta n , es decir, hace $n-1$ iteraciones.
- Cuando $i = 2$, el bucle interno j va desde 3 hasta n , es decir, hace $n-2$ iteraciones.
- Cuando $i = 3$, el bucle interno j va desde 4 hasta n , es decir, hace $n-3$ iteraciones.
- En general, el bucle interno corre $n-i$ veces.
- En la última vuelta cuando $i = n-1$, el bucle interno solo realiza 1 iteración.

Selection sort: Análisis

Por lo tanto, el número de iteraciones del bucle interno se puede expresar como la siguiente serie:

$$(n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1$$

A esta sumatoria se le conoce como **serie aritmética** y se puede definir como:

$$\sum_{i=1}^k i = \frac{k (k + 1)}{2}$$

Sustituyendo k por $(n - 1)$, tenemos que el total de iteraciones será:

$$\frac{(n - 1)((n - 1) + 1)}{2} = \frac{(n - 1)(n)}{2} = \frac{n^2 - n}{2}$$

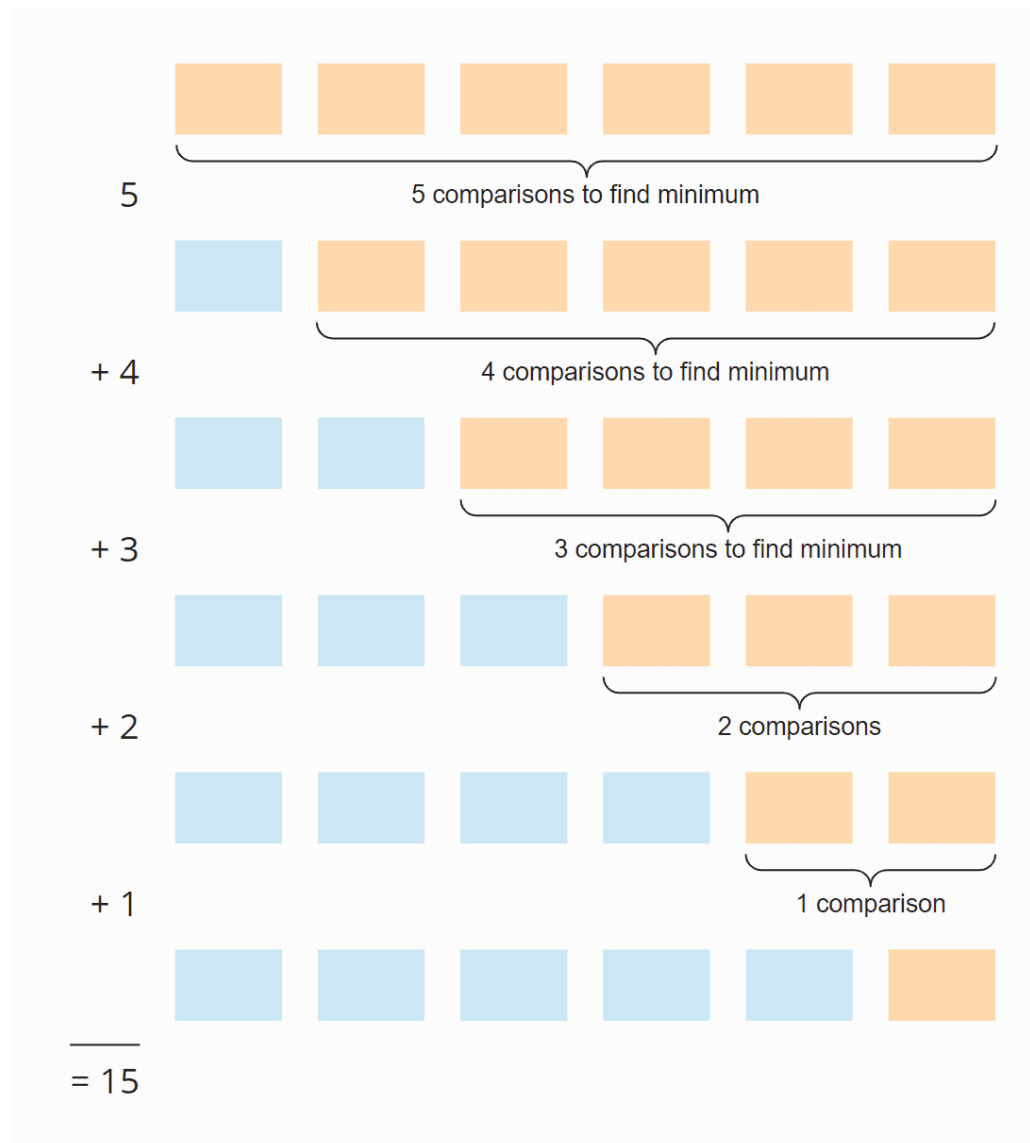
Simplificando, tenemos que la complejidad para este algoritmo será de **$O(n^2)$** .

Selection sort: Todos los casos

- Esto cubre todos los casos, ya que independientemente de si el arreglo ya está ordenado o no, el bucle interno siempre realizará las mismas comparaciones, por lo que siempre será **$O(n^2)$** .
- Esto se debe a que los bucles *for* están definidos por el tamaño n y no dependen de una condición de salida temprana (como sí ocurre en el Insertion Sort).

Selection sort: Todos los casos

- **Mejor caso:** el arreglo ya está ordenado.
 - Se realizan todas las comparaciones $O(n^2)$, pero el pivote $pos_min = j$ nunca cambia dentro del *if*.
- **Peor caso:** el arreglo está en orden inverso.
 - Se realizan todas las comparaciones $O(n^2)$ y el pivote $pos_min = j$ cambia en cada paso posible.
- **Caso promedio:** datos aleatorios.
 - Se realizan todas las comparaciones $O(n^2)$ y el pivote cambia de forma intermitente.

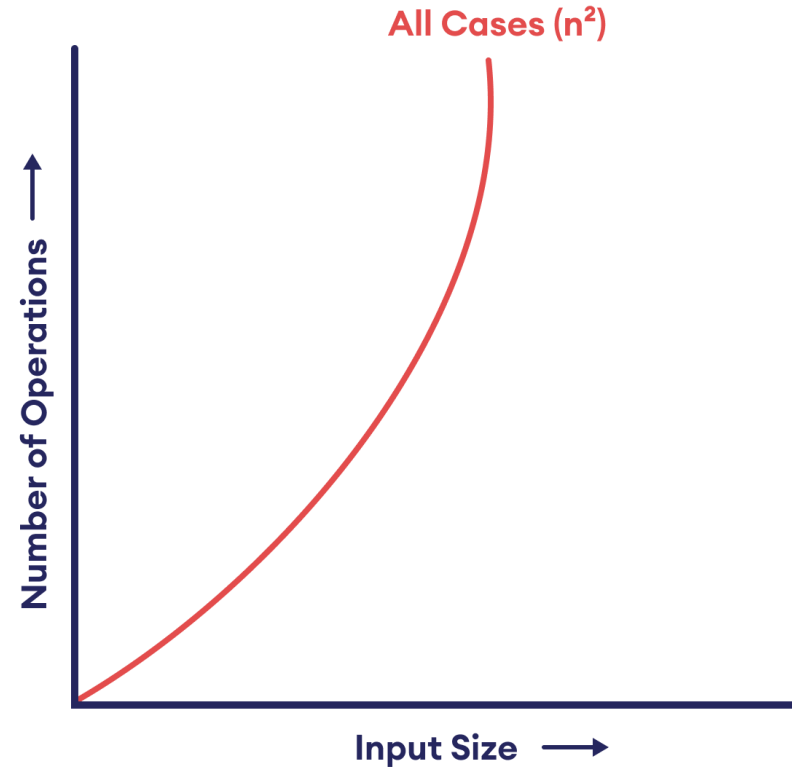


Dado que el número de comparaciones que se harán siempre será el mismo para todos los casos podemos calcularlo como: $\frac{n^2 - n}{2}$, por lo tanto para $n = 6$: $\frac{6^2 - 6}{2} = 30/2 = 15$ comparaciones

Ejemplo Selection sort

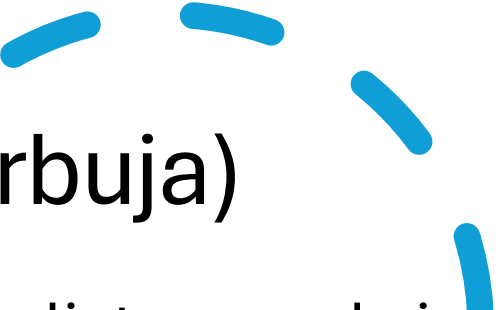
Selection sort: Resumen del análisis

- **Mejor caso:** $O(n^2)$.
- **Peor caso:** $O(n^2)$.
- **Caso promedio:** $O(n^2)$.



Bubble sort

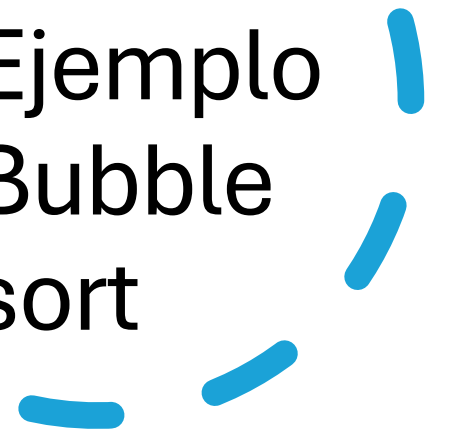
(Ordenamiento de burbuja)



- Compara cada elemento de la lista con el siguiente, intercambiándolos de posición si están en el orden equivocado.
 - Es necesario revisar varias veces toda la lista hasta que no se necesiten más intercambios.
 - Fácil de entender e implementar.
 - No es eficiente para listas grandes.
 - Casi no tiene aplicación en la vida real, más bien es usado académicamente para mostrar otra forma de ordenamiento.
- 

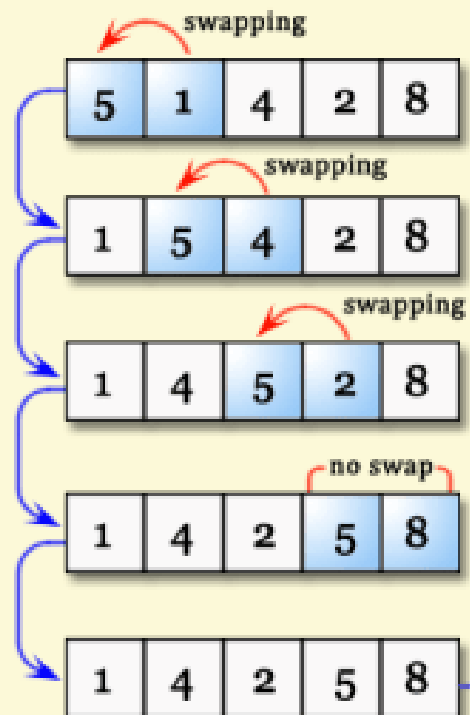
6 5 3 1 8 7 2 4

Ejemplo
Bubble
sort

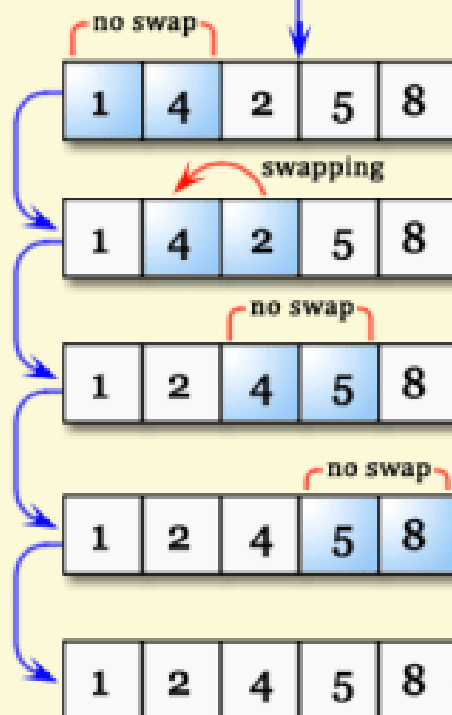


Bubble Sorting

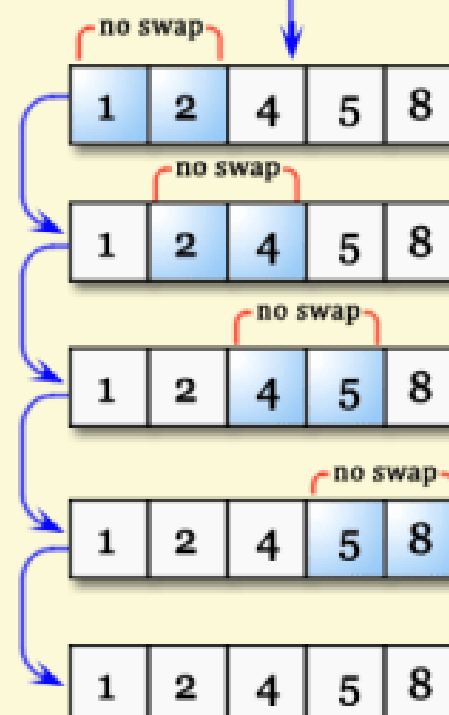
First Pass



Second Pass



Third Pass



© w3resource.com

Ejemplo
Bubble
sort

Bubble sort: pseudocódigo

Algoritmo #4

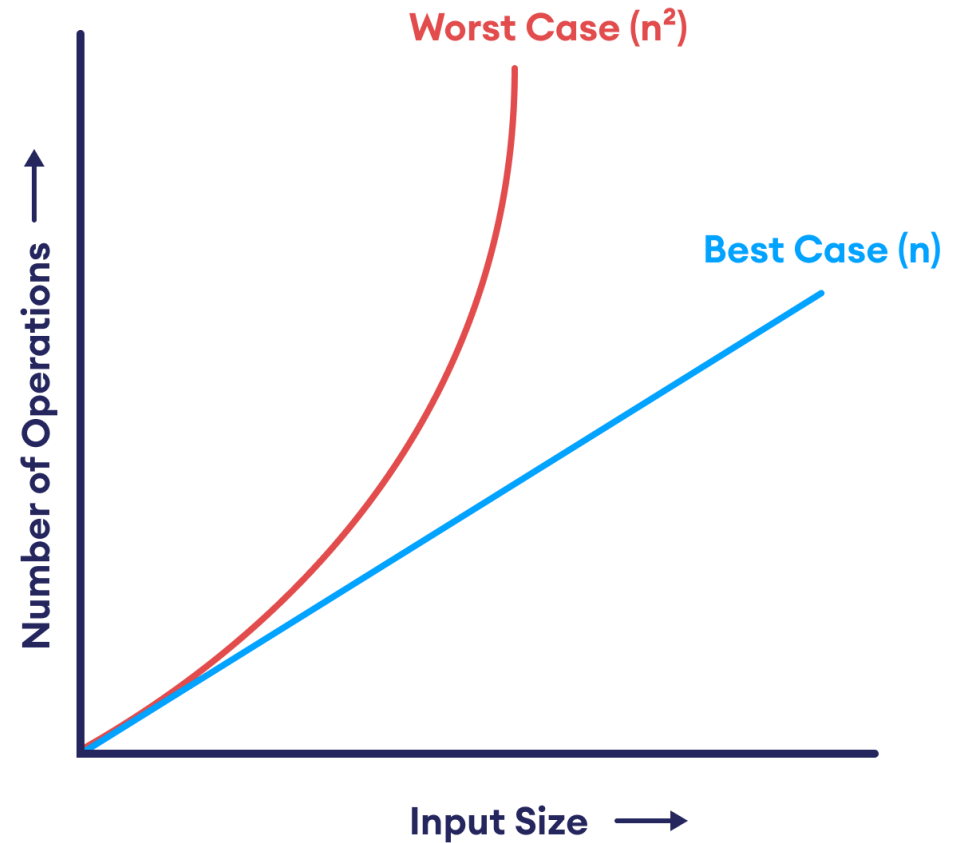
Input: $x_1, x_2, \dots, x_n$

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

```
for  $i = 1$  to  $n-1$  do                                #n-1 veces
    for  $j = 1$  to  $n-i$  do                                #n-i veces
        if  $x[j] > x[j + 1]$  then
            temp =  $x[j]$ 
             $x[j] = x[j + 1]$ 
             $x[j + 1] = temp$ 
```

Bubble sort: Resumen del análisis

- **Mejor caso:** $O(n)$
- **Peor caso:** $O(n^2)$
- **Caso promedio:** $O(n^2)$





Pausa

Repaso de algunas conceptos y herramientas
para el análisis de algoritmos recursivos

[3.1_Recursión.pptx](#)

Merge sort

(Ordenamiento por mezcla)



- Es un algoritmo de ordenamiento muy popular, reconocido por su **eficiencia** y su **estabilidad**.
 - Sigue el enfoque de diseño de ***Divide y Vencerás***.
 - Trabaja recursivamente **dividiendo** un arreglo en dos mitades, recursivamente **ordenando** las dos mitades para finalmente **combinarlas** en el arreglo ordenado.
- 

Merge sort

- **Dividir:** Divide el arreglo de n elementos en dos subarreglos de igual tamaño ($n/2$) cada uno.
- **Vencer/Conquistar:** Ordena los dos subarreglos usando recursivamente *Merge sort*.
- **Combinar:** Combina los dos subarreglos para obtener un solo arreglo ordenado.

La recursión llega a su punto más bajo cuando la secuencia a ordenar tiene una longitud de 1, en cuyo caso no hay trabajo que hacer, ya que cada secuencia de longitud 1 ya está ordenada.

Merge sort: desarrollando intuición



1. PREPARACIÓN

- Entregar a cada equipo entre **8 y 16 cartas**.
- Las cartas deben estar **desordenadas**.

2. DIVIDIR

- “Dividan el montón en **dos partes iguales**”
- Luego vuelven a dividir cada montón
- Repiten hasta que tengan montones de **1 sola carta**

3. MEZCLAR ORDENANDO

3.1. Unir pares

- “Tomen dos montones de 1 carta y júntenlos en orden”
- Ejemplo:

$[7] \text{ y } [3] \rightarrow [3, 7]$

3.2. Seguir combinando

- “Junten montones de 2 cartas en montones de 4, ordenando”
- Ejemplo:
 $[3, 7] \text{ y } [2, 6] \rightarrow [2, 3, 6, 7]$

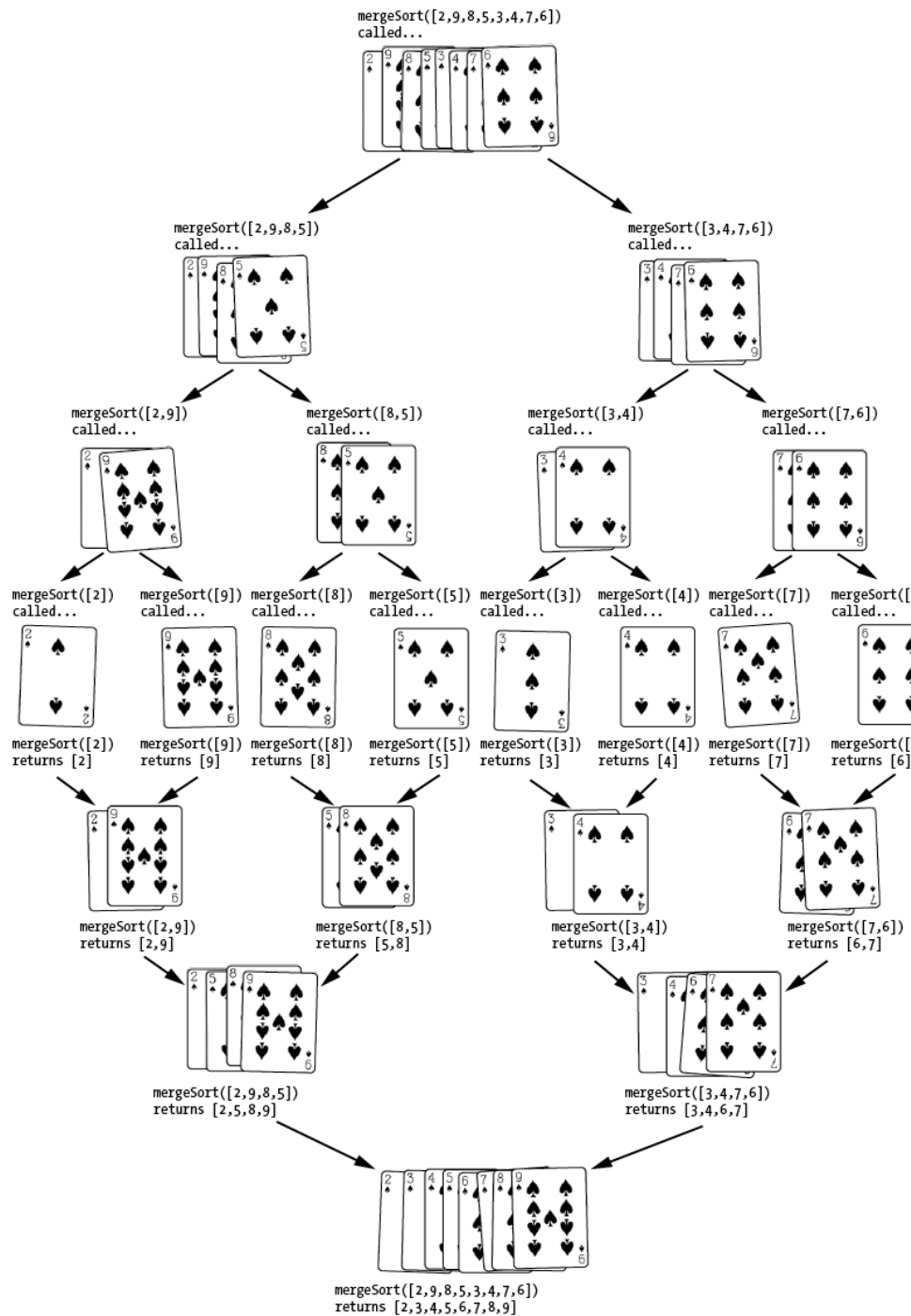
Regla: Solo pueden comparar la **primera carta de cada montón**. Van tomando la menor y la colocan en el nuevo montón.

3.3. Repetir

- Hasta volver a un solo montón.
- Siempre manteniendo el orden.

Divide into half-piles phase

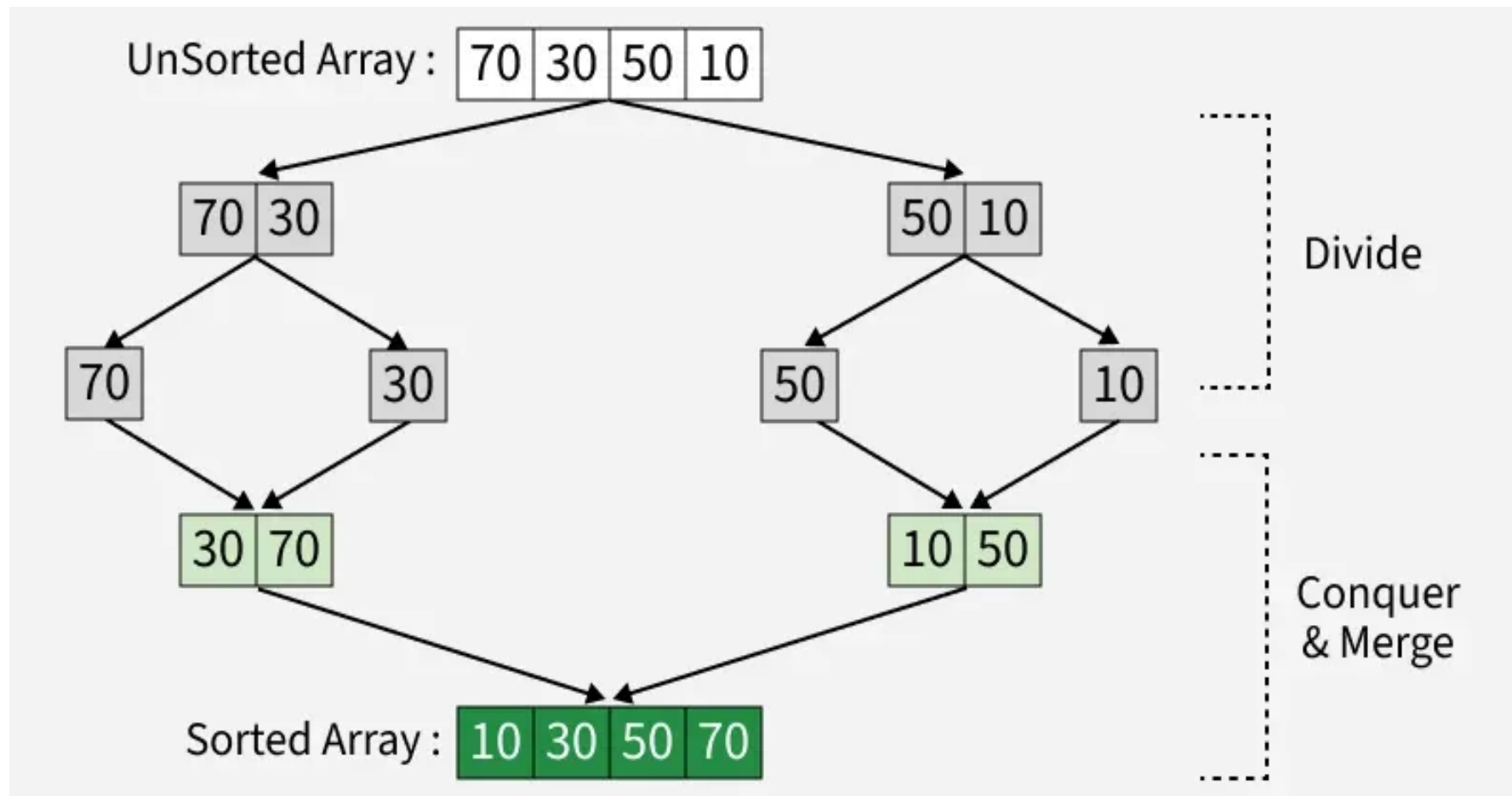
Merge into sorted-piles phase



Ejemplo 1

Merge sort

con cartas

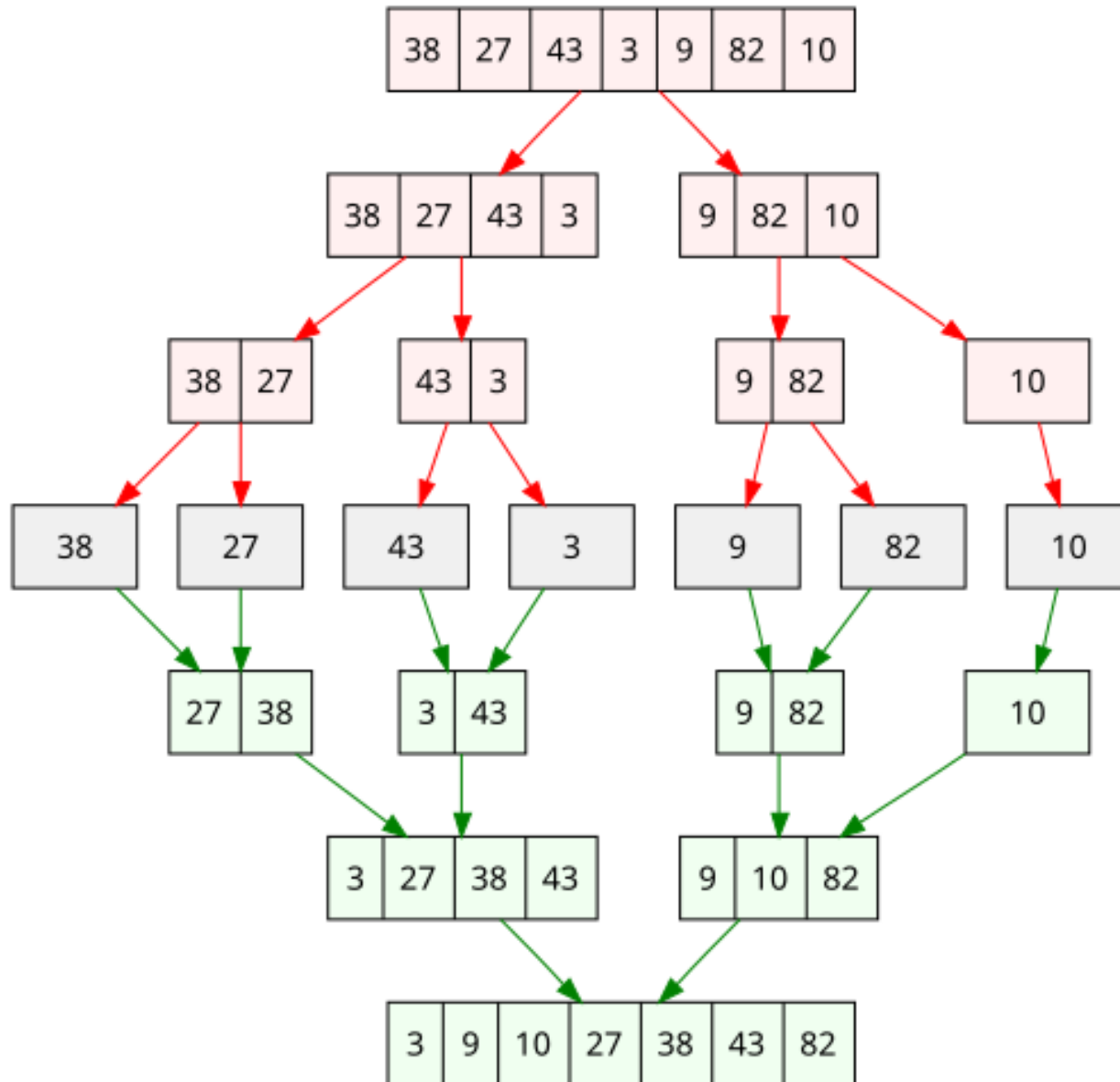


Ejemplo 2
Merge
sort

6 5 3 1 8 7 2 4

Ejemplo 3
Merge sort





Ejemplo 4
Merge sort
 n es impar

Merge sort: Procedimientos

1. MERGESORT

Dividir

2. MERGE

Ordenar y combinar/mezclar

Merge sort: Procedimiento 1

1 - MERGESORT

- Es el procedimiento principal que utiliza la recursión para ***dividir*** el problema hasta llegar a elementos individuales.
- Se llama a sí mismo para ***ordenar*** de forma independiente la mitad izquierda y la mitad derecha.
- Detiene la división cuando los sub-arreglos tienen un solo elemento (ya que un solo elemento técnicamente ya está ordenado), es decir, lleva el control del **caso base**.
- Una vez que las mitades regresan ordenadas, manda llamar a ***MERGE*** (procedimiento 2) para unir las.

Merge sort: Procedimiento 2

2 – MERGE:

- Es el encargado de **mezclar o combinar** los dos sub-arreglos ya ordenados con la finalidad de formar un solo arreglo ordenado.
- Crea dos **arreglos auxiliares Izquierda (I)** y **Derecha (D)** para almacenar temporalmente los datos de las mitades que va a mezclar.
- **Compara** los elementos frontales de las dos “pilas” mientras ambas tengan elementos disponibles. En el momento en que una de las dos mitades se agota, el bucle principal se detiene inmediatamente.
- Cuando una "pila" se acaba, la otra todavía tiene elementos que, por definición, ya están ordenados y son mayores que los que ya pusimos en el arreglo final A.

Merge sort: pseudocódigo

Algoritmo #5 Procedimiento 1 – MERGESORT

Input: $A = x_1, x_2, \dots, x_n$,

izq = índice inicial del subarreglo, **der** = índice final del subarreglo

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

MERGESORT(**A**, **izq**, **der**)

1. DIVIDE: encontrar el punto medio para partir el arreglo

Cuando $\text{izq} = \text{der}$, el subarreglo tendrá 1 elemento (ya está ordenado) por lo tanto ya no divide ni ordena

if $\text{izq} < \text{der}$ **do**
 $\text{medio} = (\text{izq} + \text{der}) / 2$ # (redondear hacia abajo)

2. VENCE/CONQUISTA: Llamadas recursivas para ordenar cada mitad

→ **MERGESORT**(**A**, **izq**, **medio**) # mitad izq

→ **MERGESORT**(**A**, **medio + 1**, **der**) # mitad der

3. COMBINA: Unir las dos mitades ordenadas

→ **MERGE**(**A**, **izq**, **medio**, **der**)

Merge sort: pseudocódigo

Algoritmo #5 Procedimiento 2 – MERGE

Input: $A = x_1, x_2, \dots, x_n$, **izq** = índice inicial del subarreglo, **der** = índice final del subarreglo, **medio** = índice final del primer subarreglo

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

3. COMBINA

MERGE(A , **izq**, **medio**, **der**)

$n1 = \text{medio} - \text{izq} + 1$

 # calcular n para arreglo Izq

$n2 = \text{der} - \text{medio}$

 # calcular n para arreglo Der

 crear arreglo $I[1..n1]$

 # crear arreglo temporal izq

 crear arreglo $D[1..n2]$

 # crear arreglo temporal der

for $i = 1$ **to** $n1$ **do**

 # copiar los datos de A al arreglo temporal I

$I[i] = A[\text{izq} + i - 1]$

for $j = 1$ **to** $n2$ **do**

 # copiar los datos de A al arreglo temporal D

$D[j] = A[\text{medio} + j]$

Merge sort: pseudocódigo

Algoritmo #5 Procedimiento 2 – MERGE

```
i = 1                                # Índice inicial para el sub-arreglo izquierdo I
j = 1                                # Índice inicial para el sub-arreglo derecho D
k = izq                              # Índice inicial para el arreglo original A

# Bucle principal: compara y mezcla mientras ambos sub-arreglos tengan elementos
disponibles
while i ≤ n1 and j ≤ n2 do
    if I[i] ≤ D[j] do
        A[k] = I[i]
        i = i + 1
    else
        A[k] = D[j]
        j = j + 1

    k = k + 1
```

Merge sort: pseudocódigo

Algoritmo #5 Procedimiento 2 – MERGE

```
# Al salir del bucle, uno de los sub-arreglos se agotó.  
# Copiar los elementos restantes de I (si los hay)  
while  $i \leq n1$  do  
     $A[k] = I[i]$   
     $i = i + 1$   
     $k = k + 1$   
  
# Copiar los elementos restantes de D (si los hay)  
while  $j \leq n2$  do  
     $A[k] = D[j]$   
     $j = j + 1$   
     $k = k + 1$ 
```

Merge sort: Análisis procedimiento MERGE

Análisis de la comparación en las “**pilas**” en *MERGE*:

- Supongamos que tenemos dos pilas de cartas boca arriba sobre una mesa. Cada pila está ordenada, con las cartas más pequeñas arriba. Queremos fusionar las dos pilas en una única pila de salida ordenada, que estará boca abajo sobre la mesa. Nuestro paso básico consiste en elegir la carta más pequeña de las dos pilas boca arriba, retirarla de su pila (lo que expone una nueva carta superior) y colocarla boca abajo sobre la pila de salida. Repetimos este paso hasta que una pila de entrada esté vacía, momento en el que simplemente tomamos la pila de entrada restante y la colocamos boca abajo sobre la pila de salida.
- Computacionalmente, las primeras operaciones del *MERGE* toman un tiempo constante $\Theta(1)$. El copiado de los datos a los arreglos temporales toma $\Theta(n_1) + \Theta(n_2)$, por lo tanto tomará un tiempo $\Theta(n)$. El bucle principal while compara una vez y mueve una vez cada elemento, por lo tanto le tomará un tiempo $\Theta(n)$. Por último, el vaciado de los elementos que hayan sobrado en alguna de las pilas tomará también $\Theta(n)$.
- Decimos que el procedimiento **MERGE** toma un tiempo de ejecución $\Theta(n)$.

Divide y Vencerás: Análisis para obtener $T(n)$

De forma general, la ecuación de recurrencia para el tiempo de ejecución $T(n)$ de un algoritmo Divide y Vencerás, se deduce de los tres pasos del paradigma básico: Dividir, Vencer y Combinar.

- Si el tamaño del problema es lo suficientemente pequeño, digamos $n \leq c$ para una constante c , entonces la solución directa requiere de un tiempo constante $\Theta(1)$.
- Por otro lado, cuando la división del problema produce a subproblemas, cada uno de tamaño $1/b$ del problema original, entonces se requiere un tiempo $T(n/b)$ para resolver un problema de tamaño (n/b) y por lo tanto tomará $aT(n/b)$ para resolverlos todos.
- Si toma $D(n)$ tiempo para dividir el problema en subproblemas y $C(n)$ tiempo para combinar las soluciones de los subproblemas en una solución al problema original, obtendremos la siguiente recurrencia:

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{de lo contrario} \end{cases}$$

Merge sort: Análisis para obtener $T(n)$

Aplicar Merge sort cuando el arreglo solo tiene 1 elemento toma un tiempo constante $\Theta(1)$.

Cuando tenemos $n > 1$ elementos, desglosamos el tiempo de ejecución de la siguiente forma:

- **Dividir:** Este paso realiza el cálculo de la mitad del subarreglo, lo cual toma un tiempo constante. Por lo tanto:

$$D(n) = \Theta(1)$$

- **Conquistar/Vencer:** Se resuelven de manera recursiva dos subproblemas de tamaño $(n/2)$, lo cual contribuye al tiempo de ejecución:

$$2T(n/2)$$

- **Combinar:** El procedimiento *MERGE* en un subarreglo de n elementos toma $\Theta(n)$. Por lo tanto,
 $C(n) = \Theta(n)$

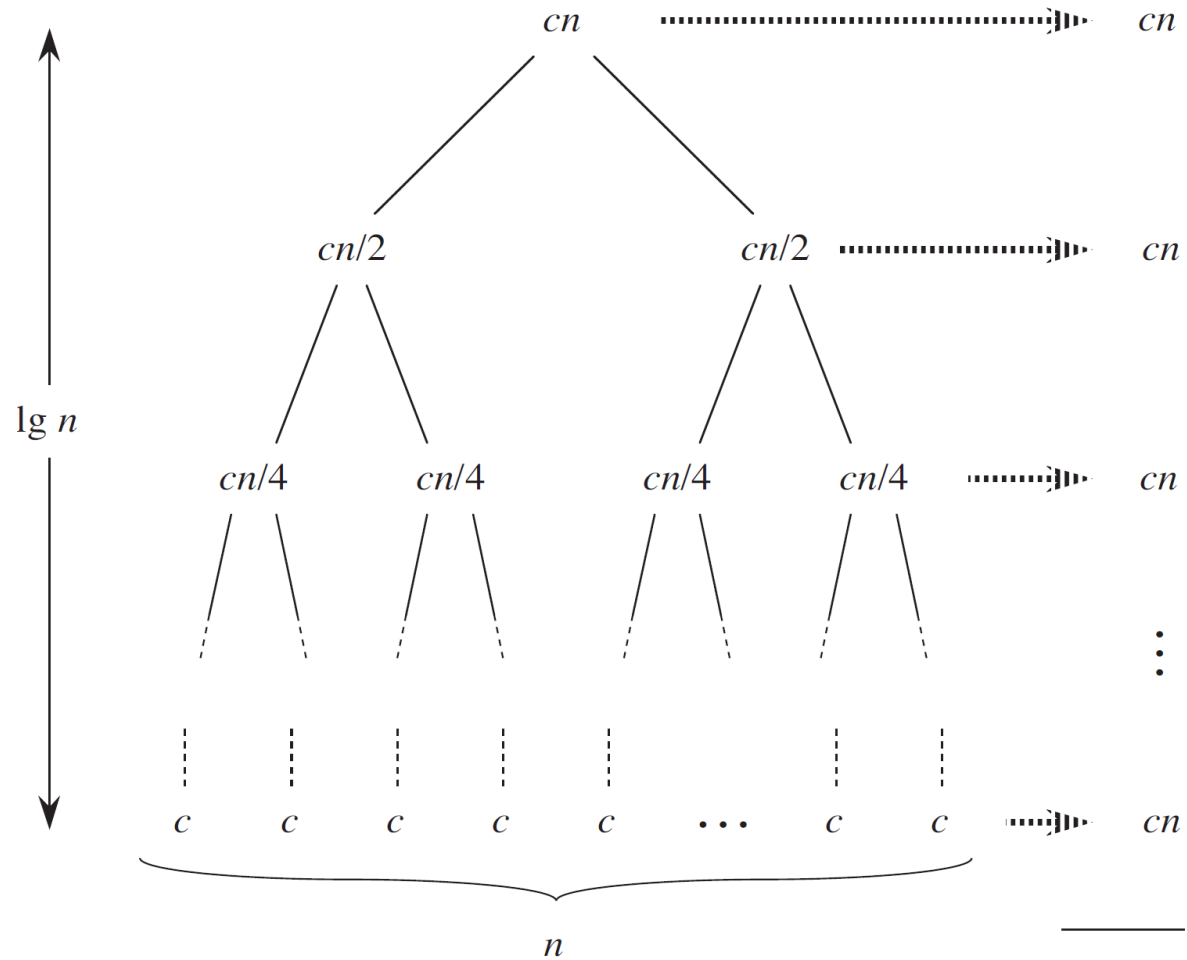
$$\underbrace{T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}} \quad \Rightarrow \quad \underbrace{T(n) = \begin{cases} c & \text{si } n = 1 \\ 2T(n/2) + cn & \text{si } n > 1 \end{cases}}$$



Resolviendo con árbol de recursión

$$T(n) = 2T(n/2) + cn$$

$$T(n) = 2T(n/2) + cn$$



(d)

Total: $cn \lg n + cn$

Para calcular el costo total simplemente sumamos los costos de todos los niveles.

El árbol de recursión tiene $\lg n + 1$ niveles, cada uno con un costo de cn , para un costo total de:

$$cn(\lg n + 1) = cn \lg n + cn.$$

Ignorando el término de menor orden y la constante c , obtenemos:

Complejidad = $O(n \log n)$



Resolviendo con teorema maestro

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

Paso 1: Identificar valores

- $a =$
- $b =$
- $f(n) =$

Paso 2: Calcular $n^{\log_b a}$

- $n^{\log_b a} =$
 $=$
 $=$

Paso 3: Comparar $f(n)$ vs $n^{\log_b a}$

$$f(n) =$$

$$n^{\log_b a} =$$

Entonces:

$$f(n) \text{ _____ que } n^{\log_b a},$$

Por lo tanto aplica el **Caso**

Paso 4: Resultado

$$T(n) = \Theta(\text{_____})$$

$$T(n) = \Theta(\text{_____})$$

Complejidad = _____

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

Paso 1: Identificar valores

- $a = 2$
- $b = 2$
- $f(n) = cn$

Paso 2: Calcular $n^{\log_b a}$

$$\begin{aligned}
 \bullet \quad n^{\log_b a} &= n^{\log_2 2} \rightarrow \log_2 2 = 1 \\
 &= n^1 \\
 &= \underline{\underline{n}}
 \end{aligned}$$

Paso 3: Comparar $f(n)$ vs $n^{\log_b a}$

$$\begin{aligned}
 f(n) &= cn \\
 n^{\log_b a} &= n
 \end{aligned}$$

Entonces:

$f(n)$ es igual que $n^{\log_b a}$,

Por lo tanto aplica el **Caso 2**

Paso 4: Resultado

$$T(n) = \Theta(n^{\log_b a} \log n)$$

$$T(n) = \Theta(n \log n)$$

Complejidad = $O(n \log n)$

Merge sort: Análisis de complejidad

- **Complejidad temporal:**

Este algoritmo presenta la misma complejidad para todos los casos, ya que siempre realiza el mismo trabajo, sin importar que tan ordenados o desordenados esté el arreglo original.

$$\text{Mejor caso} = \text{Peor caso} = \text{Caso promedio} \\ \underline{O(n \log n)}$$

Merge sort supera a otros algoritmos de ordenamiento, como *Insertion sort* $O(n^2)$.

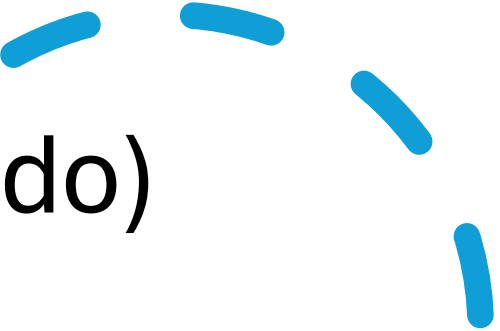
- **Respecto a la Complejidad espacial:**

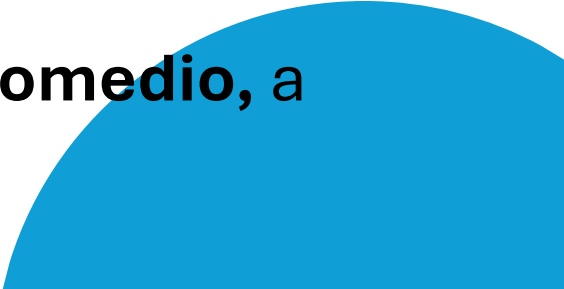
El procedimiento MERGE (combinar) no opera *in-place*, es decir, tiene que crear copias temporales de los subarreglos. Si la memoria/espacio son limitados, quizás este no sea el mejor algoritmo.

Merge sort: Resumen del análisis de complejidad

Caso	Complejidad $O()$
Mejor caso	$O(n \log n)$
Peor caso	$O(n \log n)$
Caso promedio	$O(n \log n)$

Quick sort (Ordenamiento rápido)



- Es un algoritmo de ordenamiento que, al igual que *Merge sort* sigue el enfoque de diseño de ***Divide y Vencerás*** usando ***recursión***.
 - Se destaca por su procedimiento de ***partición***, el cual reorganiza los elementos alrededor de un ***pivote*** sin necesidad de arreglos auxiliares (***ordenamiento in-place***). Esto permite que no sea necesario realizar operaciones adicionales para combinarlos al final.
 - Es uno de los favoritos por su gran **eficiencia en el caso promedio**, a pesar de que es inestable.
- 

Quick sort

- **Dividir:** Mediante la selección de un **pivote**, particiona y reorganiza el arreglo A en dos subarreglos (posiblemente vacíos) de manera que todos los elementos del primer subarreglo sean menores o iguales al pivote, y los elementos del segundo subarreglo sean mayores.
- **Vencer/Conquistar:** Ordena los dos subarreglos de manera recursiva llamando la función *Quicksort*.
- **Combinar:** Dado que los subarreglos ya están ordenados, no se necesita trabajo adicional para combinarlos.

Quick sort: desarrollando intuición



1. PREPARACIÓN

- Entregar a cada equipo entre **8 y 16 cartas**.
- Las cartas deben estar **desordenadas**.

2. DIVIDIR

Paso A. “Identifiquen la carta pivote” (última carta)

Paso B. “Coloquen un marcador i a la izquierda de la primera carta”

Paso C. “Usen otro marcador j para recorrer carta por carta, desde la primera hasta la penúltima, haciendo la siguiente pregunta:

¿Es la carta actual menor o igual al pivote?

Si la respuesta es **SÍ**:

- Muevan el marcador i una posición a la derecha
- Intercambien físicamente la carta en la posición i por la carta en la posición j .
- Muevan el marcador j una posición a la derecha

Si la respuesta es **NO**:

- No hagan nada y pasen a la siguiente carta con el marcador j .

Paso D. “Una vez que se han revisado todas las cartas excepto la última (el pivote), intercambien el pivote con la carta que esté justo a la derecha de i , es decir, $i + 1$.”

Quick sort: desarrollando intuición

3. CONQUISTAR (Recursión)

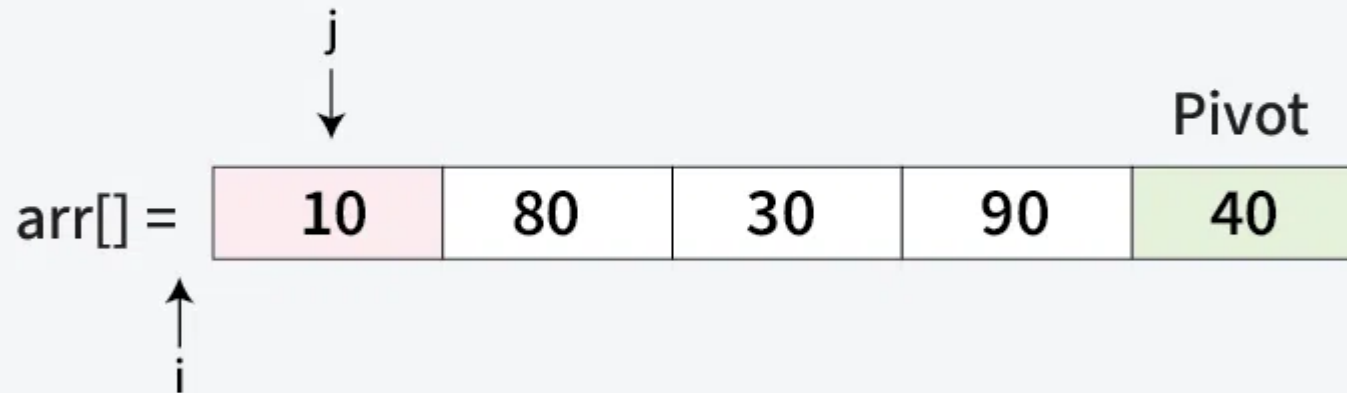
- “Repitan los pasos A, B, C y D con el sub-mazo de la izquierda y luego con el de la derecha”
- **Regla:** El pivote ya no se mueve, ya se encuentra ordenado en la posición que debe ocupar.
- **Paso Base:** Si un sub-mazo tiene solo 1 carta, ya está ordenado.

EJEMPLO: $A = [6, 1, 9, 8, 7, 2, 5, 7]$



01
Step

Pivot Selection: The last element $\text{arr}[4] = 40$ is chosen as the pivot.
Initial Pointers: $i = -1$ and $j = 0$.

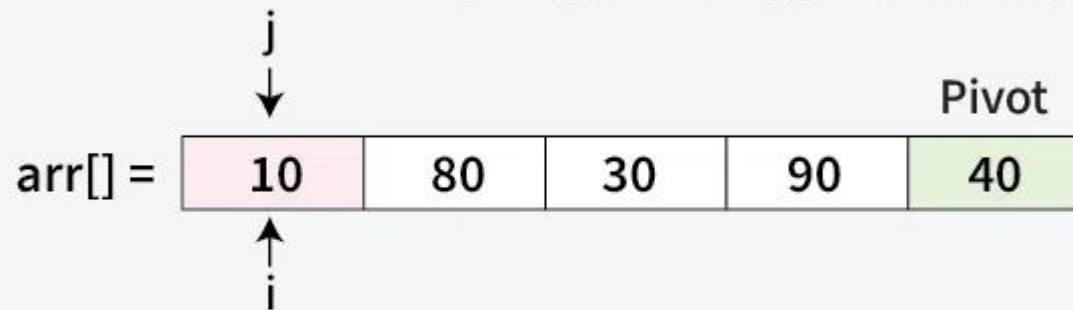


Quick sort

Ejemplo 1
Quick sort

02
Step

Since, $\text{arr}[j] < \text{pivot}$ ($10 < 40$)
Increment i to 0 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1

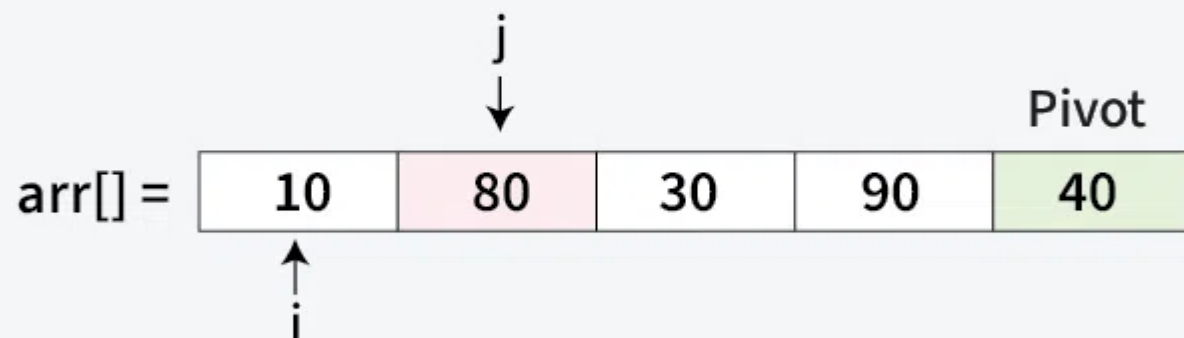


Quick sort

Ejemplo 1
Quick sort

03
Step

Since, $\text{arr}[j] > \text{pivot}$ ($80 > 40$)
No swap needed. Increment j by 1

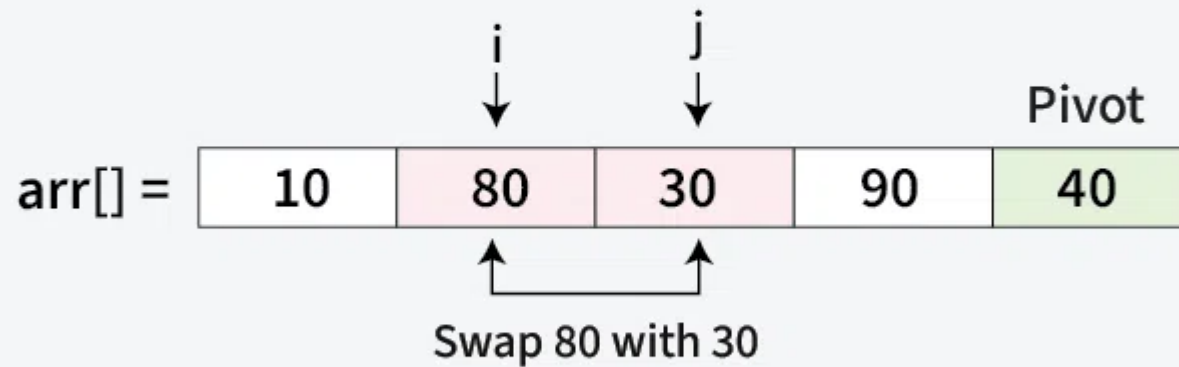


Quick sort

Ejemplo 1
Quick sort

04
Step

Since, $\text{arr}[j] < \text{pivot}$ ($30 < 40$)
Increment i by 1 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1

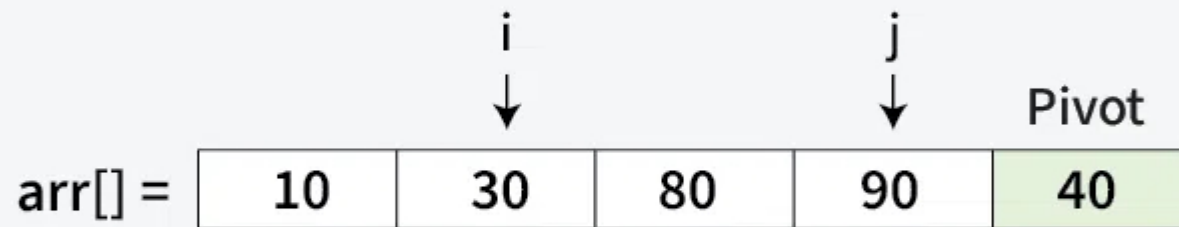


Quick sort

Ejemplo 1
Quick sort

05
Step

Since, $\text{arr}[j] > \text{pivot}$ ($90 > 40$)
No swap needed. Increment j by 1

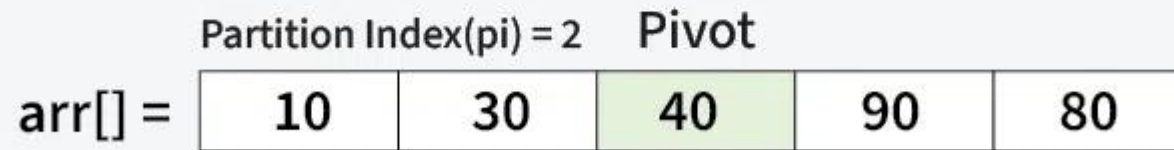
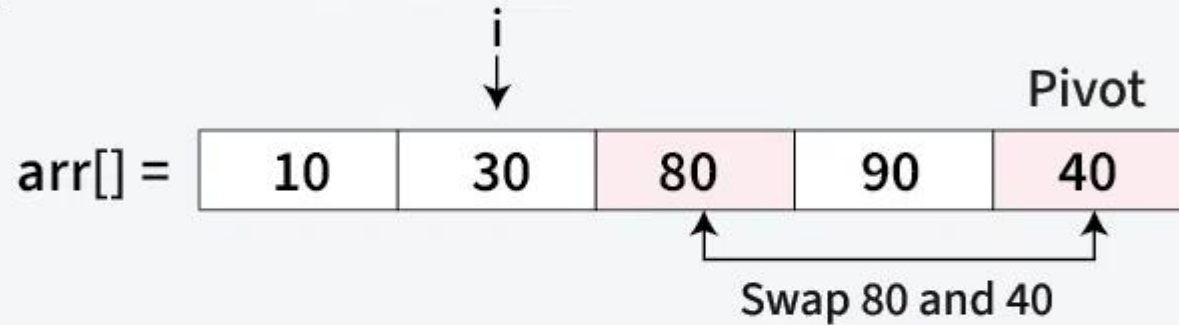


Quick sort

Ejemplo 1
Quick sort

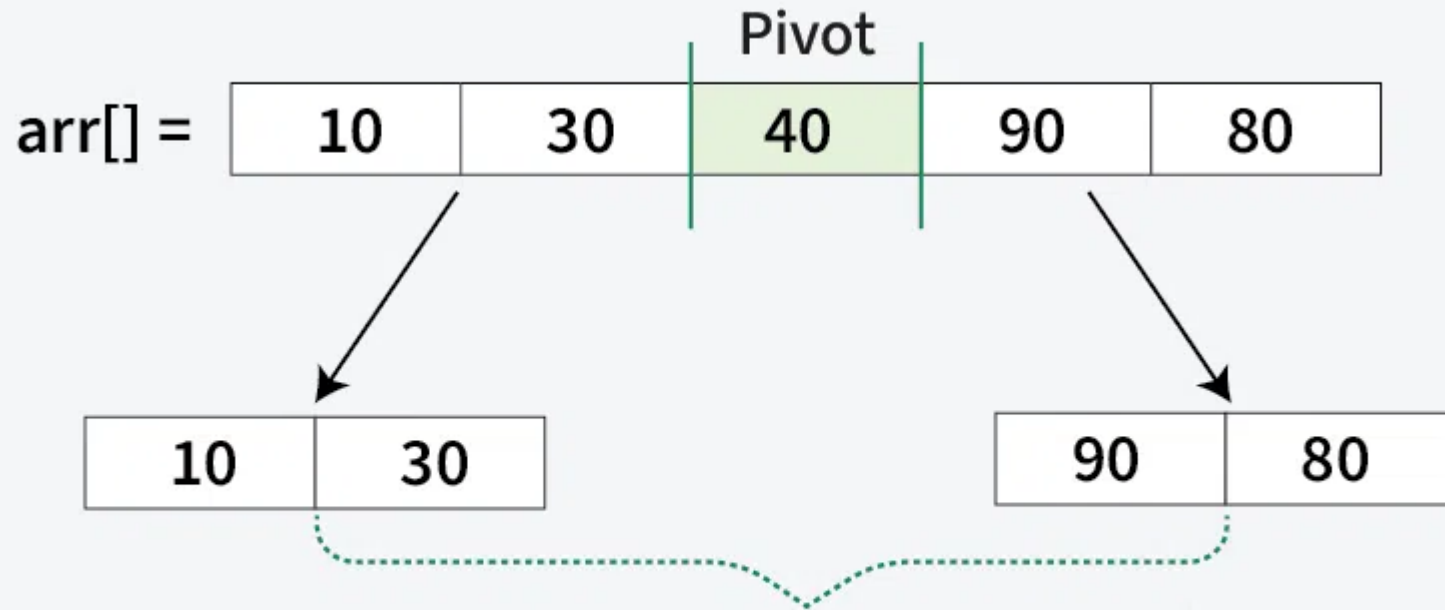
06
Step

Since traversal of j has ended. Now move pivot to its correct position, Swap $\text{arr}[i + 1] = \text{arr}[2]$ with $\text{arr}[4] = 40$.



Quick sort

Ejemplo 1
Quick sort



Recursively call quickSort for left portion (from low to $pi - 1$) and right portion (from $pi + 1$ to high) of the array to sort the elements around pivot.

Ejemplo 1
Quick sort

Quick sort: Procedimientos

1. QUICKSORT

Divide

2. PARTITION

Ordena

Quick sort: Procedimiento 1

1 - QUICKSORT

- Es el procedimiento principal que utiliza la recursión para ***dividir*** el problema hasta llegar a elementos individuales.
- Manda llamar al procedimiento ***Partition***, el cual regresa la posición del pivote (que ya se encuentra en su posición ordenada dentro del arreglo).
- Se llama a sí mismo para ***dividir y ordenar*** la mitad izquierda y la mitad derecha, a partir del pivote.
- Detiene la recursión cuando los sub-arreglos tienen un solo elemento (ya que un solo elemento técnicamente ya está ordenado), es decir, lleva el control del **caso base**.

Quick sort: Procedimiento 2

2 – PARTITION:

- Es el encargado de **seleccionar un elemento del arreglo como pivote** (el primero, el último, random, la mediana, entre otros).
- **Reorganiza los elementos del arreglo alrededor del pivote**, es decir, todos los elementos que sean menores al pivote estarán a su izquierda, y todos los elementos que sean mayores al pivote estarán a su derecha.
- **Regresa la posición del pivote**, el cual se encuentra en su posición final dentro del arreglo reorganizado. Esta posición servirá como el límite que define el inicio y fin para dividir la mitad derecha y la mitad izquierda, respectivamente.

Quick sort: pseudocódigo

Algoritmo #6 Procedimiento 1 – QUICKSORT

Input: $A = x_1, x_2, \dots, x_n$,

izq = índice inicial del subarreglo

der = índice final del subarreglo

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

QUICKSORT(**A**, **izq**, **der**)

1. DIVIDE: particionar el arreglo en dos subarreglos

Cuando $\text{izq} = \text{der}$, el subarreglo tendrá 1 elemento (ya está ordenado) por lo tanto ya no particiona ni ordena

if $\text{izq} < \text{der}$ **do**

$\text{piv} = \text{PARTITION}(\text{A}, \text{izq}, \text{der})$ # índice del pivote

2. VENCE/CONQUISTA: Llamadas recursivas para ordenar cada particion

 ➔ **QUICKSORT**(**A**, $\text{izq}, \text{piv} - 1$) # llamada recursiva en el arreglo izq

 ➔ **QUICKSORT**(**A**, $\text{piv} + 1, \text{der}$) # llamada recursiva en el arreglo der

Quick sort: pseudocódigo

Algoritmo #6 Procedimiento 2 – PARTITION

Input: $A = x_1, x_2, \dots, x_n$,

izq = índice inicial del subarreglo

der = índice final del subarreglo

Output: La secuencia ordenada de $x_1, x_2, \dots, x_n$

PARTITION(A, izq, der)

 pivot = A[der] # Selecciona pivot: último elemento del arreglo

 i = izq - 1 # índice para rastrear la posición del último elemento que sea menor o igual al pivot

 for j = izq to der - 1 # recorre el arreglo y mueve todos los elementos menores al pivot a su izq

 if A[j] ≤ pivot do

 i = i + 1

 # insertar código para intercambiar A[i] con A[j]

 # insertar código para intercambiar el pivot A[der] con A[i+1]

 return i + 1 # Regresar la posición final del pivot

Quick sort: Análisis para obtener $T(n)$

- **Combinar:** De entrada sabemos que en este paso no se realiza ninguna operación.
- **Dividir:** Este paso realiza una comparación para el control del caso base y manda llamar al procedimiento *PARTITION*, el cual ejecuta un solo ciclo y algunas operaciones de tiempo constante en cada iteración. Por lo tanto:
$$D(n) = \Theta(n)$$
- **Conquistar/Vencer:** Se resuelven de manera recursiva dos subproblemas, donde el tamaño de los subproblemas va a depender del pivote seleccionado en *PARTITION*. Por lo tanto, la complejidad temporal estará dada por el tiempo que tome ordenar el subarreglo izquierdo más el tiempo que tome ordenar el subarreglo derecho.

$$T(n) = T(n_1) + T(n_2)$$

Quick sort: **Á**nalisis de complejidad temporal – Mejor caso

El mejor caso ocurre cuando el pivote termina en el **centro** del arreglo, dividiéndolo en dos subarreglos de **tamaño no mayor a $n/2$** . Por lo tanto la ecuación de recurrencia será:

$$T(n) = 2T(n/2) + \Theta(n)$$

Resolviendo con el árbol de recursión o el teorema maestro*, obtenemos que la complejidad resulta ser la misma que Merge sort:

$$\text{Complejidad} = O(n \log n)$$

* ver diapositivas del merge sort donde resolvemos la recurrencia con ambos métodos.

Quick sort: **Á**nalisis de complejidad temporal – Peor caso

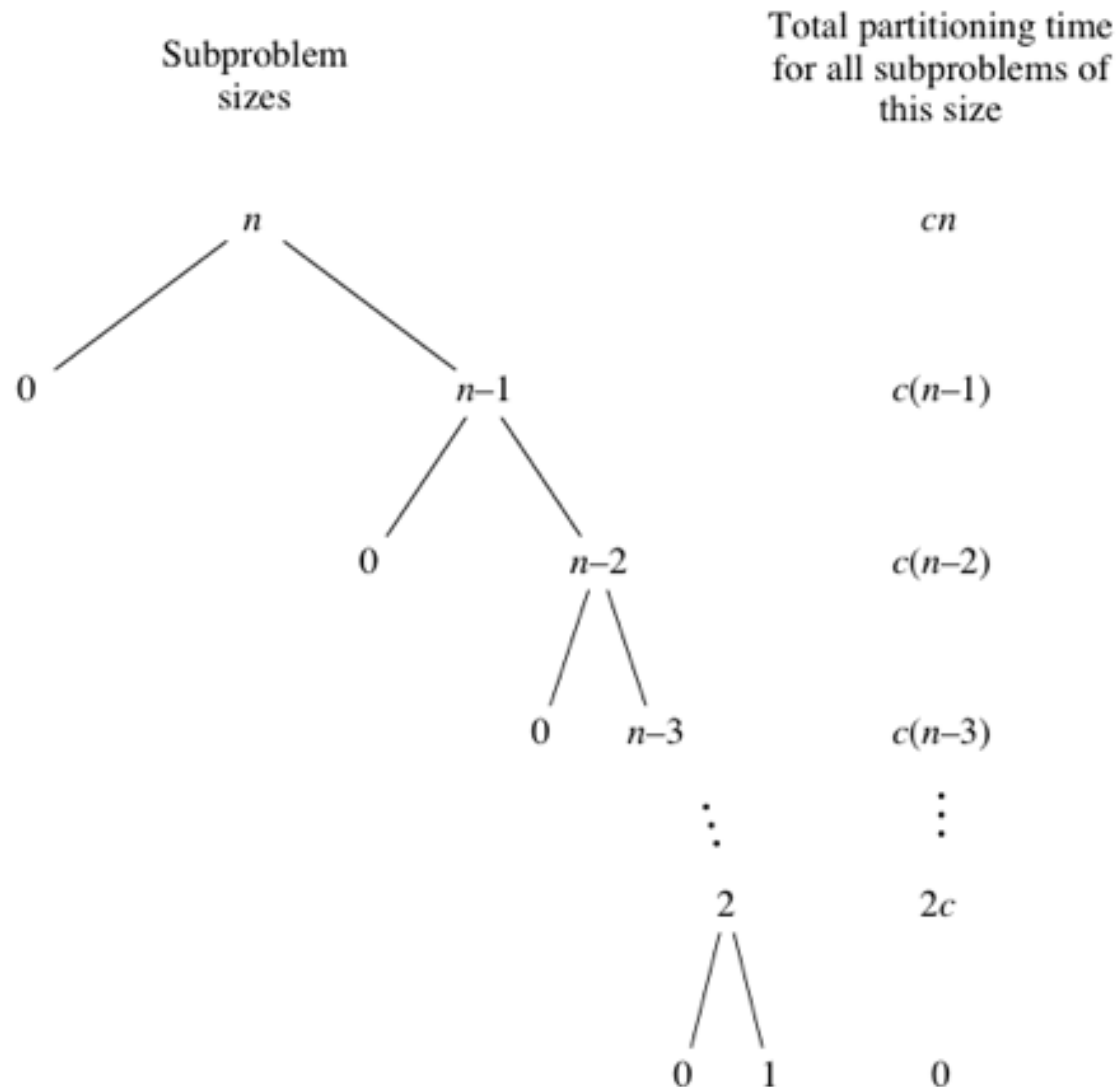
El peor caso ocurre cuando el pivote termina en un **extremo** del arreglo, es decir, siempre se elige como pivote el menor elemento (por ejemplo, en **un arreglo ya ordenado decreciente**) o el mayor elemento (por ejemplo, en **un arreglo ya ordenado ascendente**). En este caso, vamos a tener un subproblema de **tamaño $n - 1$** elementos y otro de **tamaño 0** elementos (partición desbalanceada).

Por lo tanto, la ecuación de recurrencia será:

$$\begin{aligned} T(n) &= T(n - 1) + T(0) + \Theta(n) \\ &= \underline{T(n - 1) + \Theta(n)} \end{aligned}$$

No podemos resolver con el teorema maestro, ya que no se cumple que el tamaño del problema se divida por una constante $b > 1$. Entonces resolveremos usando el árbol de recursión.

Quick sort: *Á*nalisis de complejidad temporal – Peor caso



Los costos son distintos en cada nivel. La suma de los costos ser:

$$T(n) = cn + c(n-1) + c(n-2) + \cdots + c(2) + c(1)$$

$$T(n) = c(1 + (n-1) + (n-2) + \cdots + 2 + 1)$$

Usando la **serie aritmtica**:

$$\frac{n(n+1)}{2}$$

$$\text{Costo total } T(n) = c \left[\frac{n(n+1)}{2} \right]$$

$$T(n) = \frac{c}{2} (n^2 + n)$$

$$\text{Complejidad} = O(n^2)$$

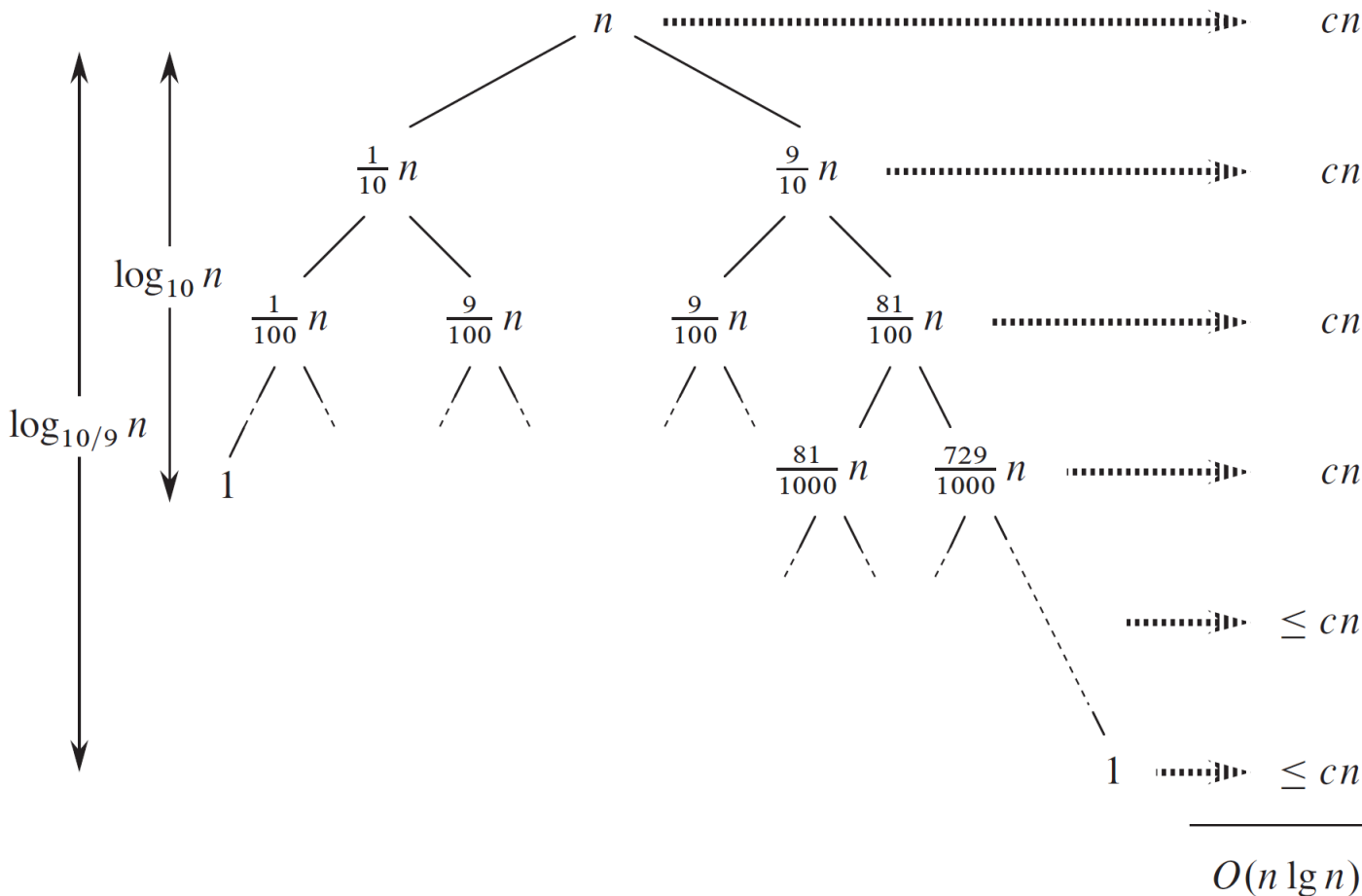
Quick sort: **Á**nalisis de complejidad temporal – Caso promedio

Este caso ocurre cuando el pivote **divide el arreglo en una proporción constante** (como 1:9), produciendo **subarreglos que, aunque no sean de igual tamaño**, mantienen un árbol de recursión de altura logarítmica.

Esto quiere decir que, se espera que **algunas particiones sean bien balanceadas y otras mal balanceadas**, las cuales van a estar distribuidas aleatoriamente en el árbol de recursión.

Para este análisis, se requiere de un análisis matemático bastante complejo, pero podemos hacer un **análisis basado en la intuición del balance**.

Quick sort: *Á*nalisis de complejidad temporal – Caso promedio



En la práctica, **es extremadamente improbable que el pivote divida siempre el arreglo de forma pésima** (0 y $n - 1$) en cada nivel de la recursión.

Incluso con **particiones constantes pero desbalanceadas** (como una división 9:1), el árbol de recursión mantiene una profundidad logarítmica. Como en cada uno de esos $\log n$ niveles el trabajo de partición es lineal (n), **el costo total se estabiliza en $n \log n$** , donde las particiones "malas" ocasionales solo añaden una constante que no afecta la complejidad asintótica.

Complejidad = $O(n \log n)$

Quick sort: Resumen del análisis de complejidad

El desempeño de este algoritmo depende principalmente de la selección del pivote. Es por ello que, la mayoría de las optimizaciones que se han hecho, se centran en la elección del pivote.

Caso	Complejidad $O()$
Mejor caso	$O(n \log n)$
Caso promedio	$O(n \log n)$
Peor caso	$O(n^2)$

Algoritmos de ordenamiento: Resumen tiempos de ejecución

Algorithm	Worst-case running time	Average-case/expected running time
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Heapsort	$O(n \lg n)$	—
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$ (expected)
Counting sort	$\Theta(k + n)$	$\Theta(k + n)$
Radix sort	$\Theta(d(n + k))$	$\Theta(d(n + k))$
Bucket sort	$\Theta(n^2)$	$\Theta(n)$ (average-case)

Bibliografía

- Lee, R.C.T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2007). Introducción al diseño y análisis de algoritmos. Un enfoque estratégico. México, D.F. Mc Graw Hill.
- Cormen T. H., Leiserson C. E., R. L. Rivest y C. S. (2022). *Introduction to Algorithms* .
- Cormen, T. H. (2013). *Algorithms Unlocked*.
- https://en.wikipedia.org/wiki/Sorting_algorithm
- https://es.wikipedia.org/wiki/Ordenamiento_por_inserci%C3%B3n
- <https://visualgo.net/en/sorting>
- <https://dipeshpatil.github.io/algorithms-visualiser/#/>
- https://es.wikipedia.org/wiki/Ordenamiento_de_burbuja
- <https://www.bigocheatsheet.com/>
- https://es.wikipedia.org/wiki/Ordenamiento_por_mezcla
- https://developer-nvidia-com.translate.goog/blog/merge-sort-explained-a-data-scientists-algorithm-guide/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sge#:~:text=Para%20implementarlo%20usted%20mismo:,una%20lista%20combinada%20y%20ordenada.
- <https://www.youtube.com/watch?v=TJyl6FAVtYk>
- https://www.youtube.com/watch?v=ACFZn_xQcz8
- <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>
- <https://www.enjoyalgorithms.com/blog/quick-sort-algorithm>